

Data Science Lab

Report on

VisionGuard – Intelligent Eye Disease Detection System

By

Kruti Bagwe	13
Tanmay Bhatkar	14
Styris Deruz	15
Anish Kalbhor	16

Mentor:

Ms Amrita Mathur
Assistant Professor



St. Francis Institute of Technology
(Engineering College)
2025-26

Index

Chapter No.	Contents	Page No.
1	Introduction	3
	1.1 Background	3
	1.2 Objectives	3
2	Architectural Details	4-5
3	Implementation/ Findings	6-10
	3.1 Dataset Details	6
	3.2 Algorithm Details	7
	3.3 Screenshots of GUI with Explanation	8-10
	3.4 Performance Metrics Details	10
4	Analysis	11-12
5	Conclusion	13

Chapter 1

Introduction

1.1 Background

Eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy are among the leading causes of vision impairment and blindness worldwide. According to the World Health Organization (WHO), over 2.2 billion people globally suffer from some form of vision disorder, many of which could have been prevented or treated if detected early. In developing countries, the lack of awareness, limited access to ophthalmologists, and the high cost of diagnostic equipment contribute to delayed diagnosis and progression of these diseases. Traditional methods for eye disease detection rely on manual interpretation of retinal images by experts, a process that is time-intensive, subjective, and prone to human error, especially when screening large populations.

With advancements in artificial intelligence and computer vision, automated systems have emerged as promising tools for medical image analysis. Deep learning, particularly Convolutional Neural Networks (CNNs), has shown exceptional capability in identifying complex patterns within medical images, making it suitable for tasks such as retinal disease detection. By training on large datasets of labeled eye images, CNN-based systems can learn to recognize subtle differences between healthy and diseased retinal structures. These models can then classify new images with a high degree of accuracy, thereby assisting healthcare professionals in diagnosis and decision-making.

The VisionGuard Intelligent Eye Disease Detection System builds upon these developments by integrating image preprocessing, data augmentation, and a robust CNN architecture to classify retinal images into multiple categories. This system aims to bridge the gap between advanced diagnostic technology and practical clinical application. Through automation, VisionGuard can significantly reduce the workload of ophthalmologists, minimize diagnostic delays, and ensure timely treatment.

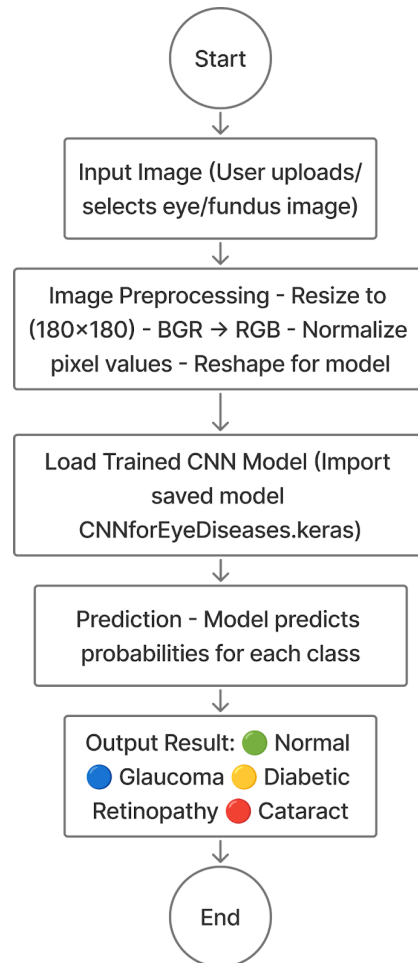
1.2 Project Objectives

- To design and implement a Convolutional Neural Network (CNN) model for automatic classification of retinal images into disease categories.
- To preprocess and augment retinal images for improving model accuracy and robustness.
- To train and validate the CNN model using a labeled dataset of normal and diseased eye images.
- To evaluate model performance using metrics such as accuracy, loss, and validation results for assessing reliability.
- To develop an intelligent detection system that assists medical professionals in early screening and diagnosis.
- To enable integration of the system with telemedicine and mobile health platforms for remote healthcare access.

Chapter 2

Architectural Details

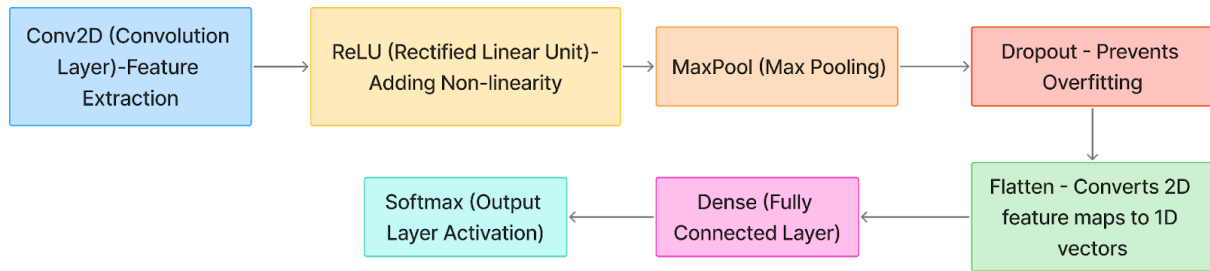
Flowchart of the system:



2.1 Flowchart of the System

The flowchart illustrates the step-by-step process of the proposed eye disease detection system using a Convolutional Neural Network (CNN) model. The system begins with the user uploading or selecting an eye/fundus image. The image undergoes preprocessing, including resizing to 180×180 pixels, conversion from BGR to RGB format, normalization of pixel values, and reshaping to match the model input. The preprocessed image is then passed to a pre-trained CNN model (CNNforEyeDiseases.keras), which predicts the probability scores for each disease class. Based on these probabilities, the system outputs the predicted result, identifying the condition as Normal, Glaucoma, Diabetic Retinopathy, or Cataract, represented by distinct color indicators for easy interpretation.

Block Diagram:



2.2 Block Diagram of the Model Training

The block diagram represents the architecture and workflow of the CNN model used for eye disease classification. The process begins with the Conv2D layer, which performs feature extraction from input images by applying convolutional filters. The extracted features are passed through the ReLU (Rectified Linear Unit) activation function to introduce non-linearity, enabling the model to learn complex patterns. The MaxPooling layer then reduces the spatial dimensions, retaining essential features while minimizing computational load. To avoid overfitting, a Dropout layer is applied, randomly deactivating neurons during training. The output feature maps are then flattened to convert 2D matrices into 1D vectors, which are fed into a Dense (Fully Connected) layer for classification. Finally, the Softmax activation function is used in the output layer to generate probability distributions across multiple eye disease classes.

Chapter 3

Implementation/Findings

3.1 Dataset Details

Dataset: <https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification>

The **Eye Diseases Classification Dataset** developed by *Gunavenkat Doddi* on Kaggle is a publicly available collection designed for multiclass image classification tasks in ophthalmology. It contains approximately **4,000 color retinal fundus images** distributed across **four major categories**:

- **Normal** – healthy retinal images without signs of disease
- **Cataract** – cloudy or opaque lens appearance affecting image clarity
- **Diabetic Retinopathy** – retinal damage caused by diabetes, often showing hemorrhages or exudates
- **Glaucoma** – optic nerve damage associated with increased intraocular pressure

Each class contains roughly **1,000 images**, providing a balanced dataset ideal for deep learning experiments. The images are aggregated from several reputable open sources such as **IDRiD**, **HRF**, and other ocular image repositories, introducing natural diversity in image quality, illumination, and resolution. This heterogeneity reflects real-world clinical variations, enabling models trained on this dataset to generalize better across imaging conditions and devices.

Key Characteristics:

- **Format:** RGB fundus images showing the posterior segment of the eye.
- **Total Images:** 4,000 (approximately 1,000 per disease class).
- **Purpose:** To support automated **eye disease detection and classification** using computer vision and deep learning.
- **Applications:** Useful for medical image classification, feature extraction, CNN training, and transfer learning tasks.

Preprocessing and Usage:

Before training, all images were **resized and normalized** for consistency across the dataset. To further enhance model robustness, **data augmentation techniques** such as *random rotation*, *contrast adjustment*, *zoom*, and *cropping* were applied. These augmentations help simulate real-world variations like changes in lighting, camera angle, and focus, ensuring the model remains invariant to such factors during inference.

Significance:

The dataset provides a realistic foundation for developing **AI-assisted diagnostic systems** that can detect and classify common eye disorders early, aiding ophthalmologists in screening large populations efficiently. Its balanced nature and variety make it a valuable benchmark for evaluating classification models in medical imaging research.

3.2 Algorithm Details

The Convolutional Neural Network (CNN) architecture implemented for this study follows a sequential deep learning pipeline designed for multiclass eye disease classification. It processes retinal fundus images through a series of convolutional, pooling, dropout, and dense layers to extract spatial features, reduce dimensionality, and perform classification into the four target classes: *Cataract*, *Diabetic Retinopathy*, *Glaucoma*, and *Normal*. The overall flow can be summarized as:

Conv + ReLU $\rightarrow$ MaxPooling $\rightarrow$ Dropout $\rightarrow$ Flatten + Dense $\rightarrow$ Softmax (4 classes)

1. Model Overview

The model is built using TensorFlow Keras Sequential API, which allows for layer-wise stacking and clear visualization of the feature extraction process. It begins with data augmentation to artificially expand the training dataset and make the model more invariant to common variations such as rotation, contrast, and scale. This step ensures better generalization when tested on unseen images.

2. Convolutional Layers

The CNN employs four convolutional blocks, each using the Conv2D operation with decreasing filter sizes ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8$).

- The first convolutional layer uses 64 filters of size (5×5) with *ReLU activation* and "same" padding. This layer captures low-level features such as edges, textures, and boundaries within the retinal structure.
- The subsequent layers (32, 16, and 8 filters respectively) progressively capture more abstract, high-level features like blood vessel structures, optic disc regions, or pathological textures relevant to each eye condition.
- The ReLU (Rectified Linear Unit) activation introduces non-linearity, enabling the network to learn complex decision boundaries while preventing the vanishing gradient problem.

Each convolutional layer is followed by a MaxPooling2D layer, which reduces the spatial dimensions of feature maps by selecting the most prominent features in each region. Pooling helps minimize computation, reduces overfitting, and provides translational invariance, which is especially beneficial for images with slight positional variations.

3. Dropout and Flatten Layers

After the final convolution and pooling stages, a Dropout layer with a rate of 0.2 is applied. Dropout randomly disables 20% of neurons during training, reducing overfitting by preventing the network from relying too heavily on specific activations. This regularization step is crucial given the dataset's moderate size ($\sim 4,000$ images).

The Flatten layer converts the 3D feature maps from the convolutional section into a 1D vector, preparing it for fully connected (dense) layers that perform the final classification.

Flattening bridges the transition from spatial feature extraction to categorical decision-making.

4. Dense Layers and Output

The flattened feature vector passes through a Dense layer of 50 neurons with ReLU activation, which enables the network to combine the learned spatial features into higher-level patterns.

Finally, a Dense output layer with 4 neurons and Softmax activation is used to classify the input into one of the four eye disease categories. Softmax converts the outputs into probability distributions summing to one, allowing the model to confidently predict the most likely class.

5. Model Compilation

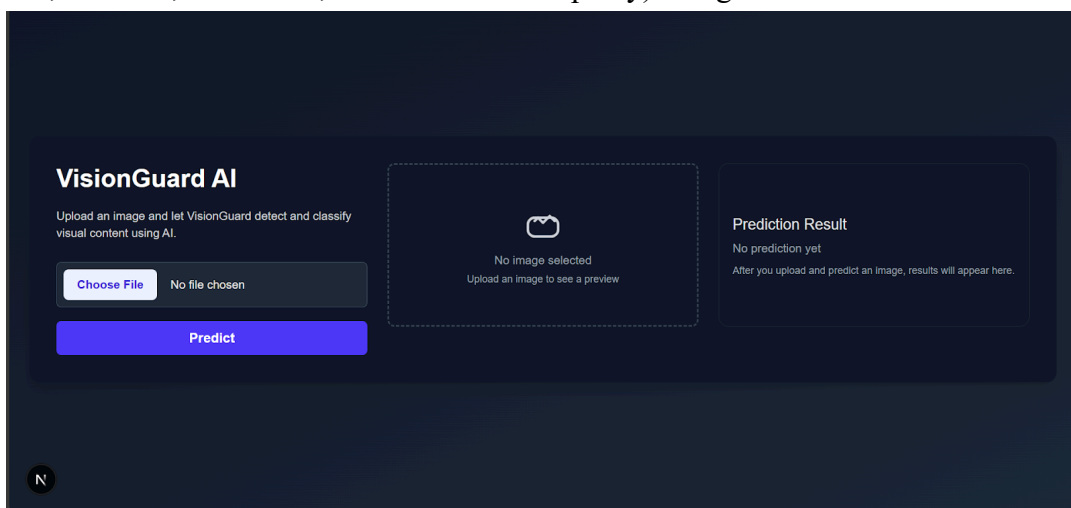
The model is compiled using the Adam optimizer, known for its adaptive learning rate and computational efficiency. The loss function used is *sparse categorical cross-entropy*, suitable for multiclass classification tasks where target labels are integer-encoded. The evaluation metric is accuracy, providing an intuitive measure of performance.

6. Summary and Significance

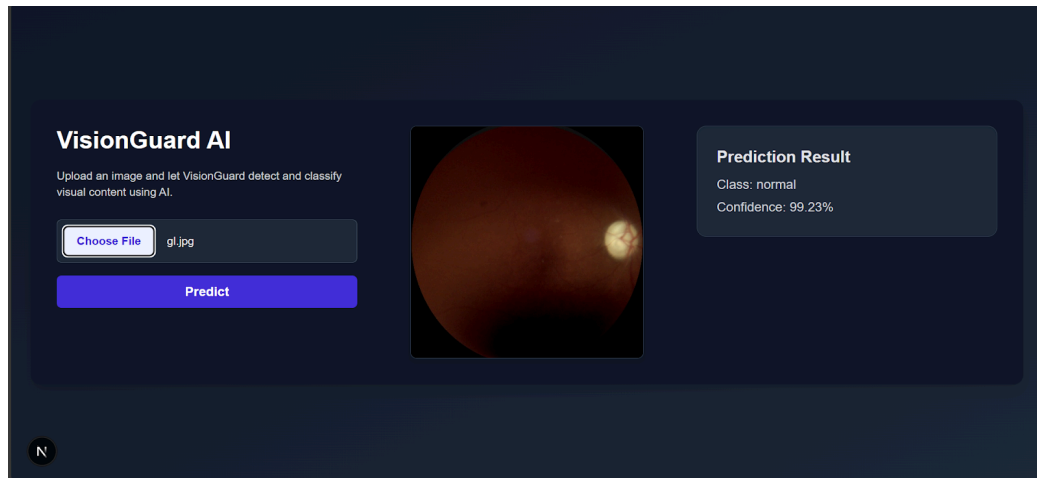
This CNN pipeline efficiently combines feature extraction (Conv + ReLU), dimensionality reduction (Pooling), regularization (Dropout), and classification (Dense + Softmax) to identify subtle differences between eye disease classes. The layered structure ensures that both local and global retinal patterns are captured, enabling the model to differentiate diseases with overlapping visual characteristics. Through careful design and balanced complexity, this CNN architecture achieves reliable classification performance while minimizing overfitting — making it well-suited for medical image analysis and diagnostic automation in ophthalmology.

3.3 Screenshots of GUI with Explanation

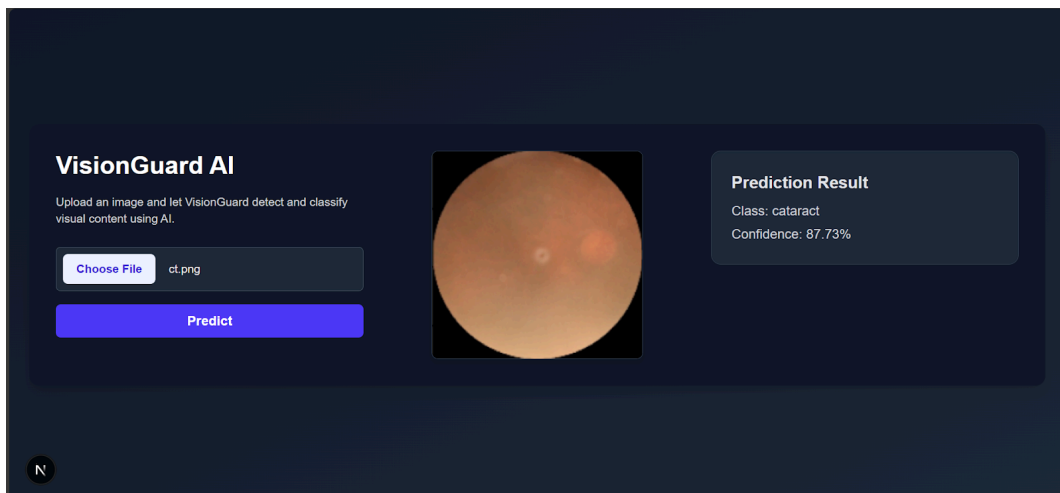
The frontend of the system was developed using Next.js, providing a fast, responsive, and user-friendly interface for eye disease prediction. The interface allows users to upload a retinal image directly from their device, which is then sent to the trained CNN model for analysis. Once the prediction is complete, the application displays the detected eye condition (Normal, Cataract, Glaucoma, or Diabetic Retinopathy) along with the confidence level.



3.3.1 Image upload interface of the VisionGuard system



3.3.2 Prediction results displayed after image upload.



3.3.3. Prediction results displayed after image upload.

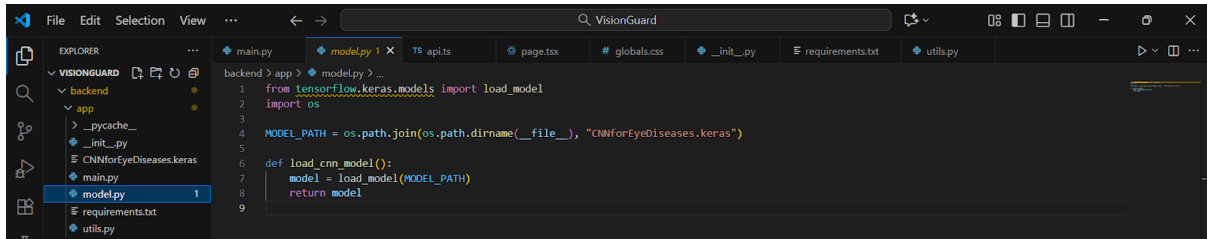
The backend is built with FastAPI, serving a CNN-based eye disease classifier. It exposes a '/predict' endpoint that accepts image uploads, preprocesses them, and returns the predicted disease class with confidence. CORS is enabled to allow requests from a Next.js frontend running on localhost:3000.

```

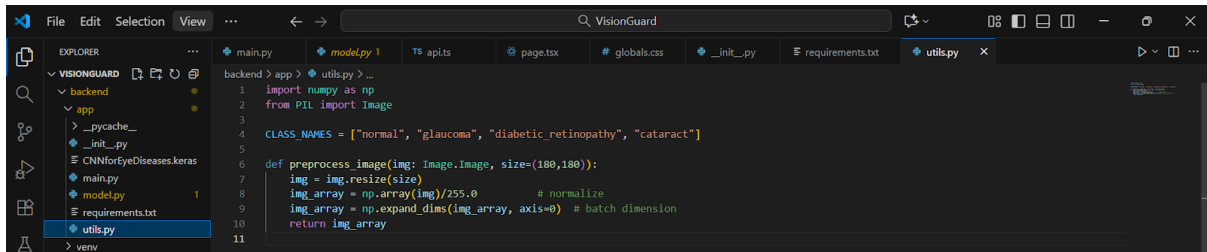
File Edit Selection View ... VisionGuard
EXPLORER
  VISIONGUARD
    backend
      app
        __pycache__
        __init__.py
        CHIMedEyeDiseases.keras
        main.py
        model.py
        requirements.txt
        utils.py
    frontend
      next
        images
          favicon.ico
          globals.css
          layout.tsx
          page.tsx
          node_modules
          public
          services
          gtag.js
          eslintrc.config.mjs
          next-env.d.ts
          next.config.ts
          package-lock.json
          package.json
          postcss.config.mjs
          README.md
    OUTLINE
    TIMELINE
  main.py
    1 from fastapi import FastAPI, UploadFile, File
    2 from fastapi.middleware.cors import CORSMiddleware
    3 from PIL import Image
    4 from io import BytesIO
    5 import numpy as np
    6 from .model import load_cnn_model
    7 from .utils import preprocess_image, CLASS_NAMES
    8
    9 app = FastAPI()
    10
    11 # Allow frontend requests
    12 app.add_middleware(
    13     CORSMiddleware,
    14     allow_origins=["http://localhost:3000"], # frontend dev URL
    15     allow_methods=["*"],
    16     allow_headers=["*"],
    17 )
    18
    19 model = load_cnn_model()
    20
    21 @app.post("/predict")
    22 async def predict(file: UploadFile = File(...)):
    23     contents = await file.read()
    24     img = Image.open(BytesIO(contents)).convert("RGB")
    25     x = preprocess_image(img)
    26     preds = model.predict(x)
    27     predicted_class = CLASS_NAMES[int(np.argmax(preds))]
    28     confidence = float(np.max(preds))
    29     return {"class": predicted_class, "confidence": confidence}
    30
    31 @app.get("/")
    32 def read_root():
    33     return {"message": "VisionGuard API is running"}
    34

```

3.3.4 main.py: FastAPI backend with /predict endpoint.



3.3.5 model.py: Loads the CNN eye disease model.



3.3.6 utils.py: Preprocesses images for model input.

3.4 Performance Metrics Details

The training process ran for 150 epochs. The plots show the training and validation accuracy and loss over these epochs. The model achieved its best validation accuracy of 0.8493 at epoch 112. The final evaluation on the test set (cell xGvvGlv_YeU6) resulted in an accuracy of approximately 0.8379 and a loss of 0.4806.

The plots indicate that the training accuracy and loss continue to improve throughout the 150 epochs, while the validation accuracy plateaus and the validation loss starts to increase after epoch 112. This suggests that the model might be starting to overfit the training data after epoch 112. Despite this, the model demonstrates good performance on the test set, with an accuracy of over 83%. This indicates that the CNN model is capable of classifying eye diseases with reasonable accuracy.

Chapter 4

Analysis

The previous pipeline used Softmax in the first convolution layer and Sigmoid in the dense layer, which limited feature extraction and caused slower convergence, resulting in a lower accuracy of around 78%. The new pipeline replaced these with ReLU activations in all convolution and dense layers and applied Softmax only at the output, leading to better non-linearity handling, faster learning, and an increased overall accuracy of 83.79%.

The model achieved-

Training Accuracy: **85.17%**

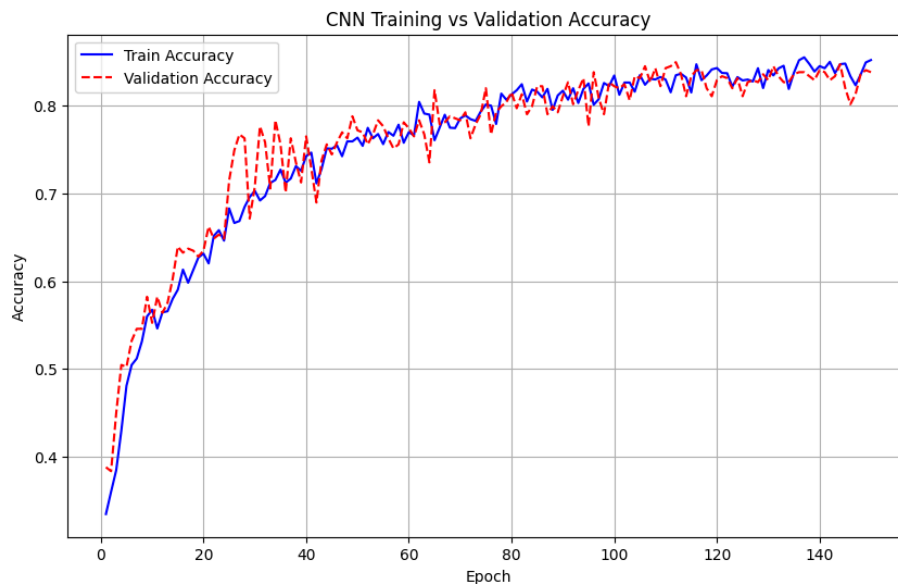
Validation Accuracy: **84.93%**

Test Accuracy: **83.79%**

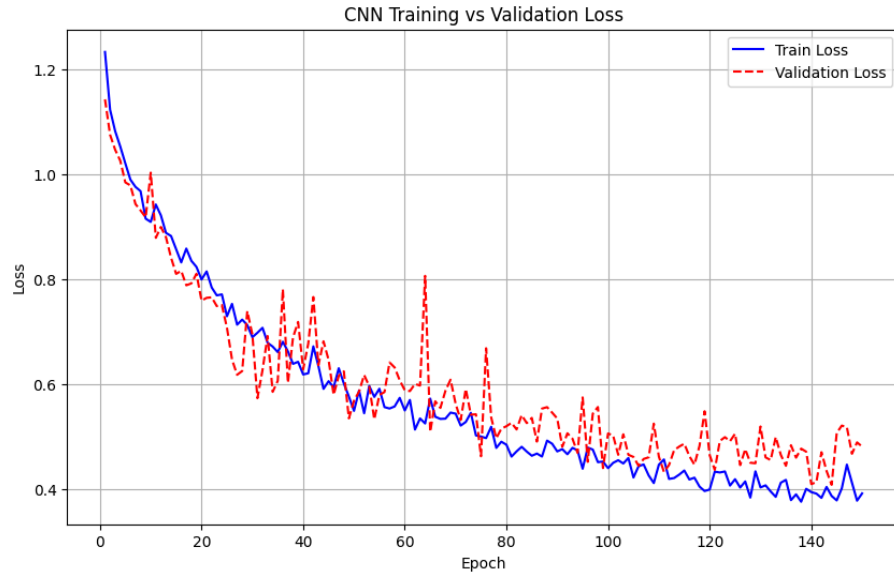
Test Loss: **0.48**

The CNN model 'CNNforEyeDiseases.keras' achieved a training accuracy of 85.17%, Validation Accuracy of 84.93% and a test accuracy of 83.79%, demonstrating good generalization performance with minimal overfitting. The model recorded a test loss of 0.48, indicating effective learning and stable convergence. The small difference between training and test accuracies suggests that the model successfully captured the key discriminative features from retinal fundus images without overfitting to the training data.

Visualization of Training Progress



4.1 Accuracy Vs Epochs Graph



4.2 Loss Vs Epochs Graph

The training and validation accuracy curves (Fig. 4.1) show a steady increase and converge after around 20 epochs. The validation accuracy closely follows the training curve, confirming stable learning behavior.

Similarly, the loss curves (Fig. 4.2) show a consistent decline without divergence between training and validation losses, indicating that the model is not overfitting.

Chapter 5

Conclusion

5.1 Conclusion

The Eye Disease Classification using Convolutional Neural Networks project effectively illustrates how deep learning can be used in the analysis of medical images. Normal, Glaucoma, Cataract, and Diabetic Retinopathy are the four primary categories into which the system can classify retinal images after training a CNN model on a dataset of eye diseases.

CNNs can efficiently extract features and learn intricate visual patterns that are challenging to detect manually, as demonstrated by the trained model's high accuracy in identifying disease patterns from eye images.

By automating the initial screening procedure, this project lessens the workload for ophthalmologists and makes it possible to detect diseases early, which is essential for preventing vision loss.

5.2 Future Work

1. **Dataset Expansion:**
Increase the dataset size and include more diverse and high-resolution retinal images to improve the model's generalization and robustness.
2. **Model Optimization:**
Experiment with advanced architectures like VGG16, ResNet, or EfficientNet for higher accuracy and better feature extraction.
3. **Explainable AI Integration:**
Implement Grad-CAM or other visualization techniques to highlight affected areas in the eye image, providing explainable results to doctors.
4. **Multi-Disease Prediction:**
Extend the system to detect other retinal or corneal disorders, making it a more comprehensive ophthalmic diagnostic tool.
5. **Real-Time Screening Integration:**
Integrate the model with retinal imaging devices for real-time detection and screening in clinics and rural healthcare centers.